

Texture in OpenGL

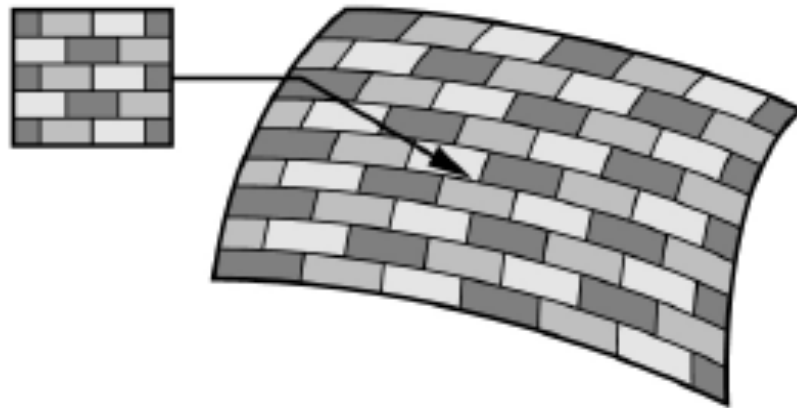
Computer Graphics 2025/2026

Problema

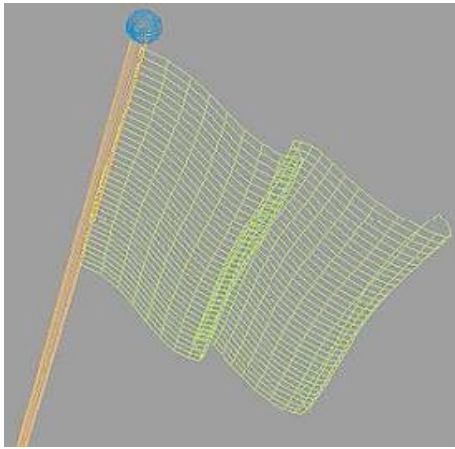
Simulare i dettagli di una superficie senza aumentare il numero di poligoni del modello.

Soluzione

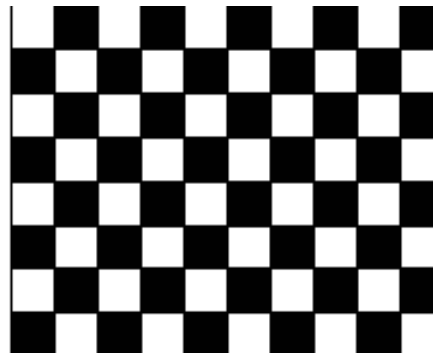
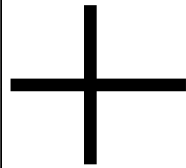
Applicare una immagine su una superficie,
come una decalcomania: **Texture Mapping**



Texture Mapping



geometria 3D
(mesh di quadrilateri)



RGB texture 2D
(color-map)



Texture Mapping



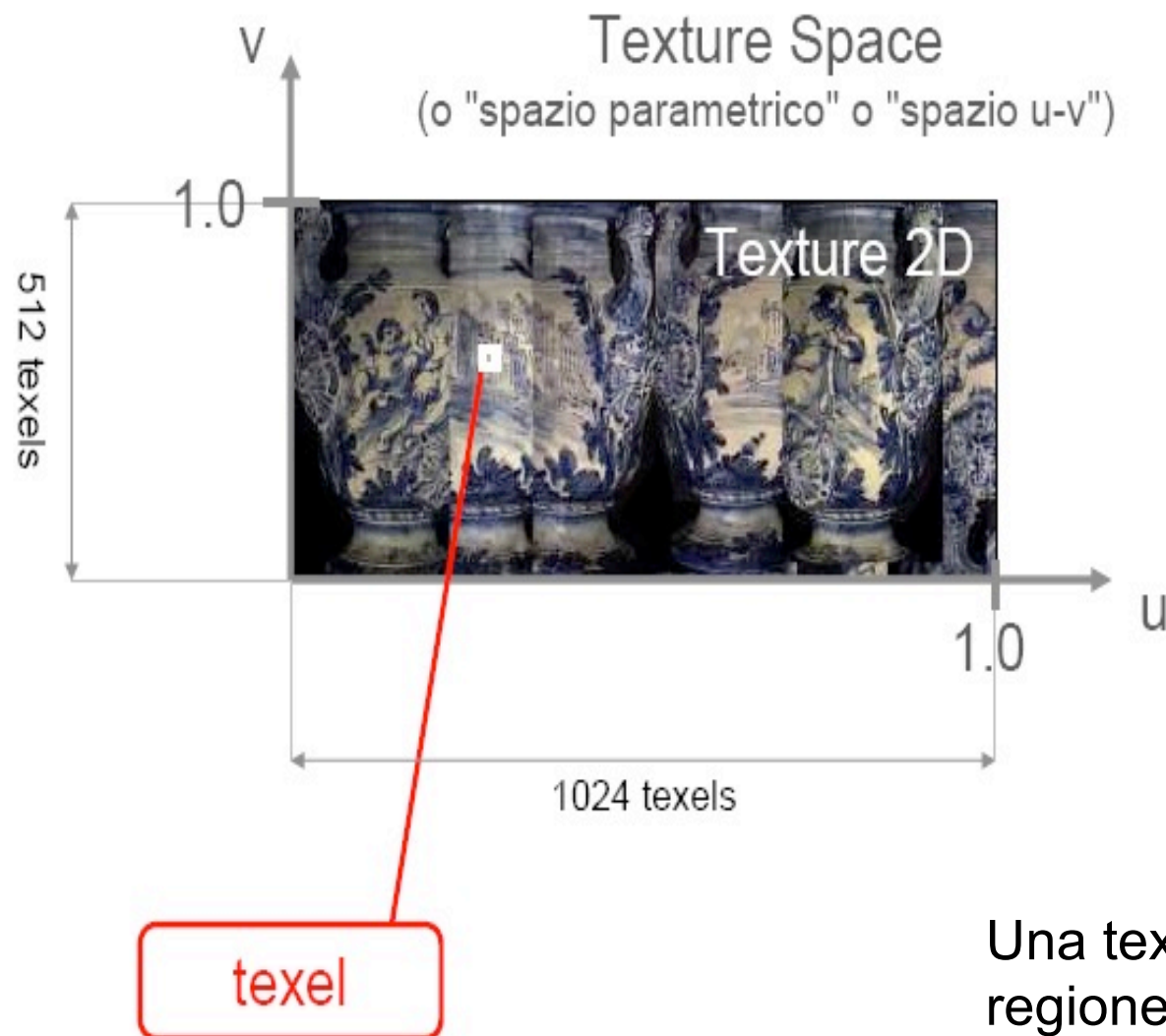
+



=



Texture Mapping



Una texture è definita nella regione $[0,1] \times [0,1]$ di uno spazio parametrico

Texture Map

- Una **texture map** è una matrice bidimensionale di dati indirizzati da due coordinate s e t comprese tra 0 ed 1 (!)
- Il dato contenuto è tipicamente un colore (la mappa è una immagine), ma potrebbe essere qualcos'altro.
- Più in generale una texture map può contenere qualunque tipo di informazione che incide sull'apparenza di una superficie: la texture map è una tabella ed il **texture mapping** consiste nel recuperare dalla tabella (lookup) l'informazione che serve per effettuare il rendering di un certo punto.

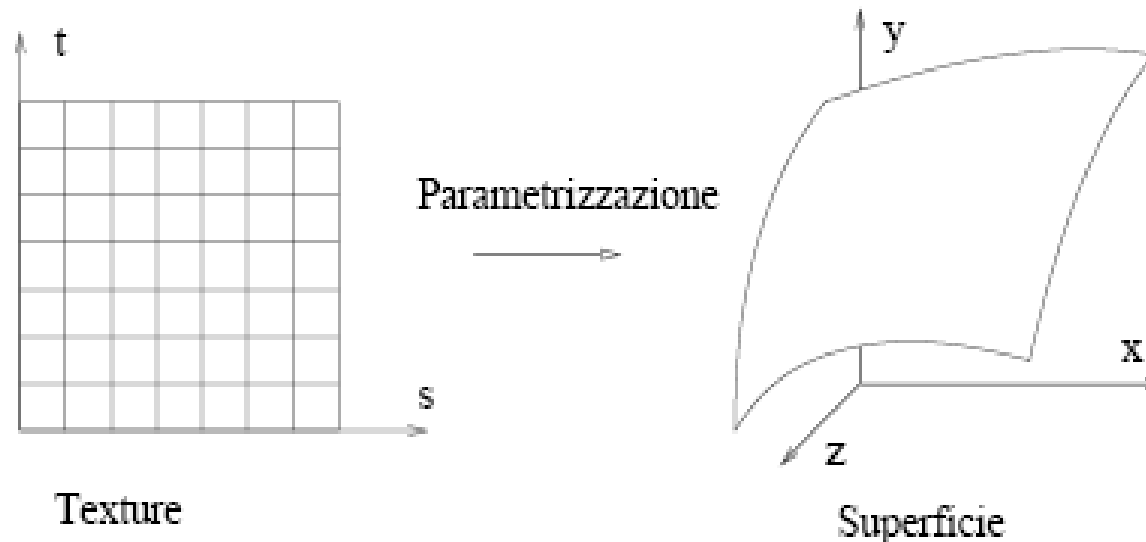
Applicare la Texture

La texture può essere applicata dopo il calcolo della illuminazione con Phong (per modificare attributi come il colore, la luminosità o la trasparenza) oppure può modificare i parametri (come le normali) che entrano nel modello di Phong.

Stabilire una corrispondenza univoca tra superficie
dell'oggetto e texture

Funzione di parametrizzazione

Occorre definire la funzione di parametrizzazione W che ad ogni punto P della superficie dell'oggetto 3D associa un punto $(s; t)$ della texture
(è una funzione che “spalma” la texture sulla superficie).



Superfici Parametriche

- Se la superficie è data in forma parametrica, ad ogni suo punto P sono associate due coordinate (parametri) $(u; v)$.
- Per ottenere W basta specificare la mappa che trasforma $(u; v)$ sulla superficie in $(s; t)$ sulla texture.

Ad esempio si consideri un cilindro di altezza h per il quale si definisce la funzione

$$W : (\theta, z) \rightarrow (s, t) = \left(\frac{m}{2\pi} \theta, \frac{n}{h} z \right)$$

dove $(m; n)$ sono le dimensioni della texture map.

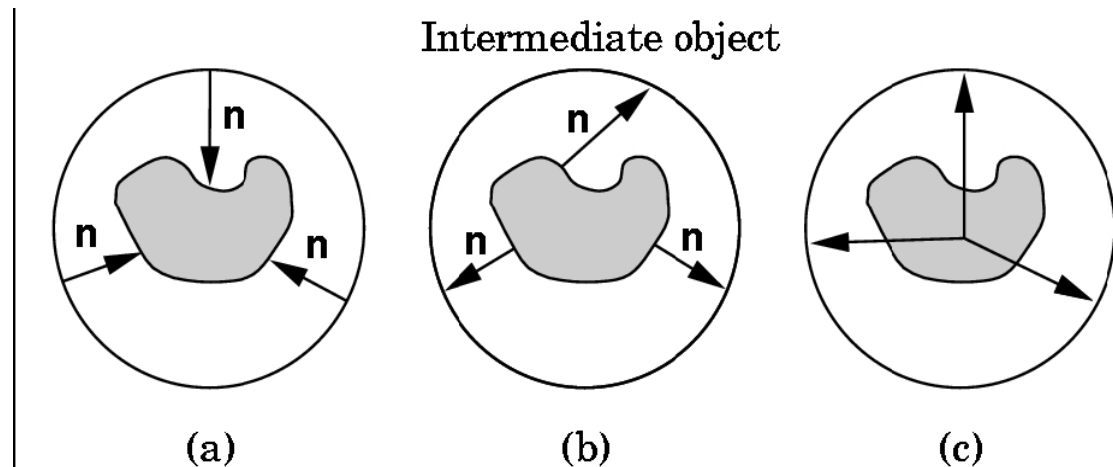
Superfici Qualsiasi

Mappatura in due passi:

- 1) Si mappa la texture su una superficie intermedia semplice, in modo che la parametrizzazione (corrispondenza punti-superficie con pixel-texture) sia immediata; questa mappatura prende il nome di *S-mapping*.
- 2) Si mappa ogni punto della superficie intermedia in un punto della superficie in esame; questa mappatura prende il nome di *O-mapping*.

La concatenazione dei due mapping genera la corrispondenza W tra i pixel della texture ed i punti dell'oggetto.

Superfici Qualsiasi



Esempi di O-mapping:

(a) Usando la normale alla superficie intermedia.

(b) Usando la normale dalla superficie dell'oggetto.

(c) Usando i raggi dal centro dell'oggetto.

Texture Mapping in OpenGL

Specificare la texture.

- glTexImage (accetta solo immagini di dimensioni potenza di 2)
- gluScaleImage

Indicare come la texture deve essere applicata in ogni pixel

- glTexEnv
- glTexParameter

Abilitare il texture mapping.

- glEnable

Disegnare la scena immettendo sia le coordinate geometriche , sia le *coordinate texture*

- glTexCoord

Specificare una Texture 2D

- `void glTexImage2D( GLenum target, GLint level, GLint internalformat, GLsizei width, GLsizei height, GLint border, GLenum format, GLenum type, const GLvoid *pixels)`

- dove:
- ***target*** il tipo di texture
 - ***level*** livello di risoluzione (è utilizzato per specificare multipli livelli di dettaglio)
 - ***internalformat*** formato di memorizzazione interno della texture (deve essere 1, 2, 3, 4, o una delle costanti simboliche)
 - ***width*** larghezza dell'immagine texture (deve essere $2^m + 2 * (border)$ per un qualche intero m)
 - ***height*** altezza dell'immagine texture (deve essere $2^m + 2 * (border)$ per un qualche intero m)
 - ***border*** margine (deve essere 0 o 1)
 - ***format*** formato dei dati pixel
 - ***type*** tipo dei dati pixel
 - ***pixels*** puntatore all'immagine in memoria

Parametri utilizzati in questo corso

Per caricare le immagini in memoria utilizzeremo una routine chiamata `read_texture` il cui codice è da scaricare dal Team del corso.

Questa routine carica un'immagine RGBA, quindi:

- ***target:* GL_TEXTURE_2D**
- ***internalformat:* GL_RGBA**
- ***format:* GL_RGBA**
- ***type:* GL_UNSIGNED_BYTE**

Scalare una immagine texture

- GLint **gluScaleImage**(GLenum *format*, GLsizei *widthin*, GLsizei *heightin*, GLenum *typein*, const void **datain*, GLsizei *widthout*, GLsizei *heightout*, GLenum *typeout*, void **dataout*)

dove: -***format*** il formato dei dati pixel
-***widthin***, ***heightin*** le dimensioni dell' immagine sorgente da scalare
-***datain*** il puntatore all' immagine sorgente
-***widthout***, ***heightout*** le dimensioni per l'immagine scalata
-***dataout*** il puntatore dove deve essere memorizzata l'immagine scalata
-***typein*** e ***typeout*** il data type per ***datain*** and ***dataout***, rispettivamente

Impostare le coordinate texture

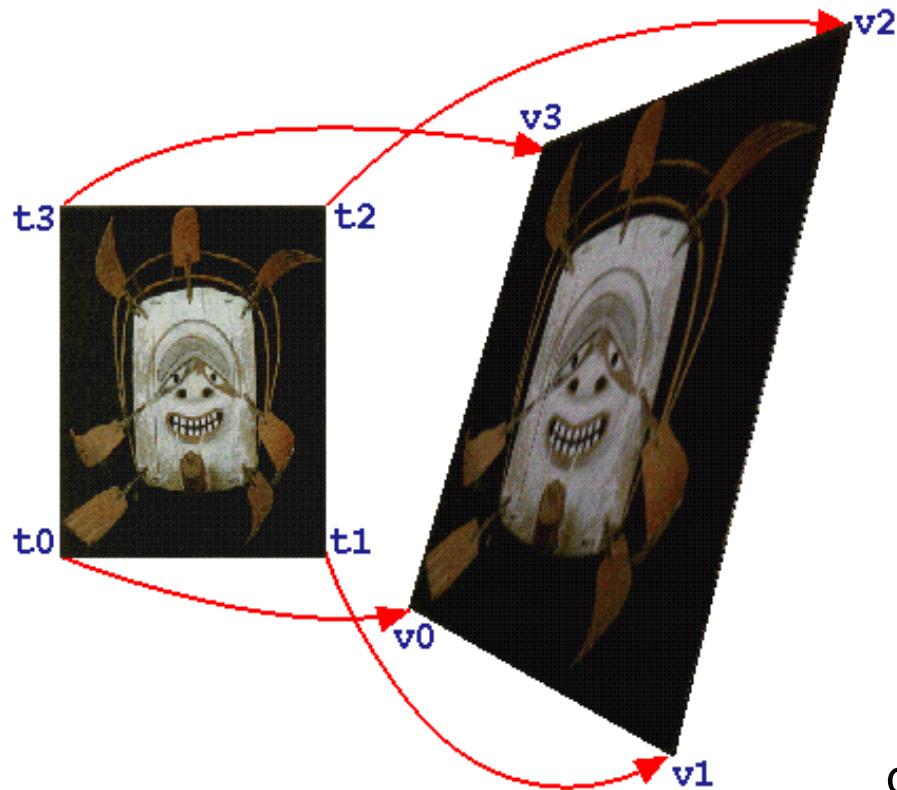
- `void glTexCoord{1234}{sifd v}(T coords);`

Utilizzando le texture 2D, le coordinate texture:

- sono due (`glTexCoord2f`)
- hanno valori compresi tra 0.0 e 1.0
- possono essere generate automaticamente
- sono interpolate tra i vertici

Esistono dei filtri per modificare l'interpolazione

Esempio



```
glBegin( GL_QUADS );  
    glTexCoord2fv( t0 ); glVertex3fv( v0 );  
    glTexCoord2fv( t1 ); glVertex3fv( v1 );  
    glTexCoord2fv( t2 ); glVertex3fv( v2 );  
    glTexCoord2fv( t3 ); glVertex3fv( v3 );  
glEnd();
```

Problema

Ogni texture mappata su di un poligono è tipicamente deformata rispetto alle trasformazioni imposte alla geometria

- La texture può essere stata allungata oppure accorciata
- Raramente un texel corrisponde esattamente ad un pixel sullo schermo

Si ha aliasing ogni qualvolta non ci sia corrispondenza uno-a-uno tra elementi del display (pixel) ed elementi della texture (texel).
Distinguiamo due casi:

Magnification: un texel corrisponde a molti pixel.

Minification: un pixel corrisponde a molti texel.

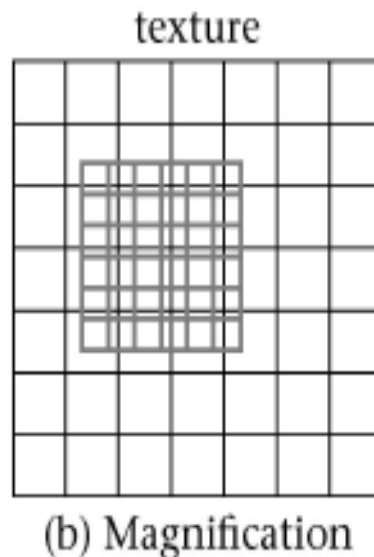
Magnification

Pixel -> Mapping -> Texel

Cosa accade se molti pixel vengono proiettati su pochi (o anche un solo) texel?

Magnification: la texture deve essere ingrandita per avere più texel.

Un texel corrisponde a molti pixel. In questo caso un singolo texel appare come un blocco di pixel nell'immagine. Quello che si vede è una versione ingrandita “a blocchi” della texture.



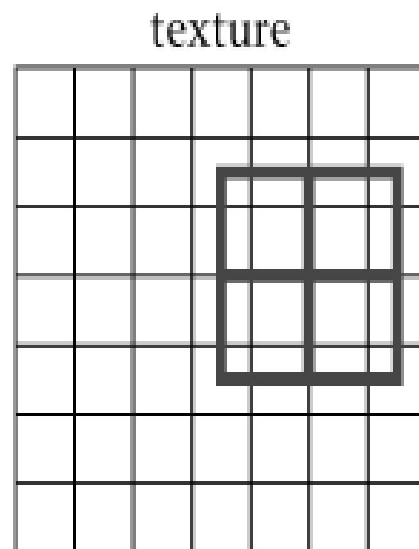
Minification

Pixel -> Mapping -> Texel

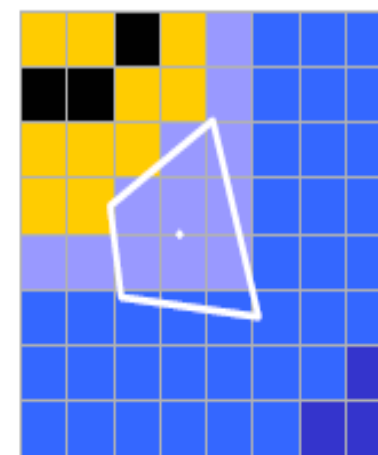
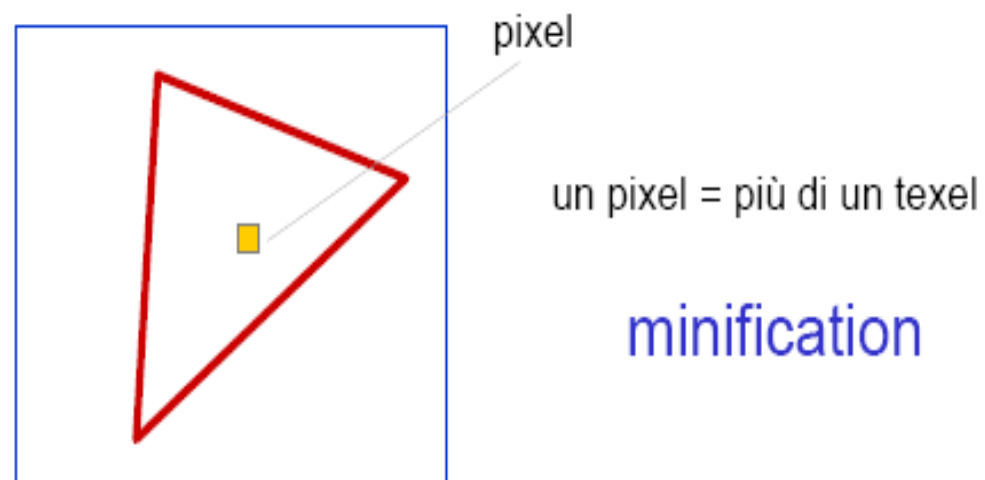
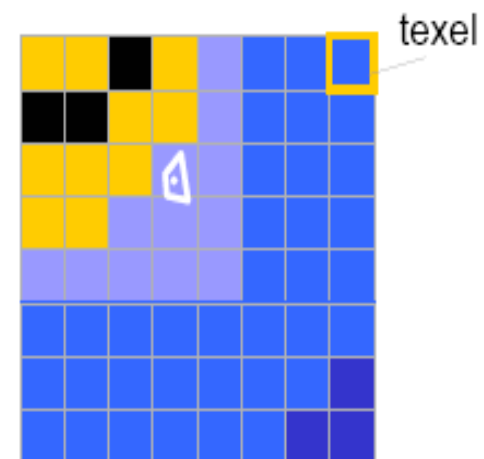
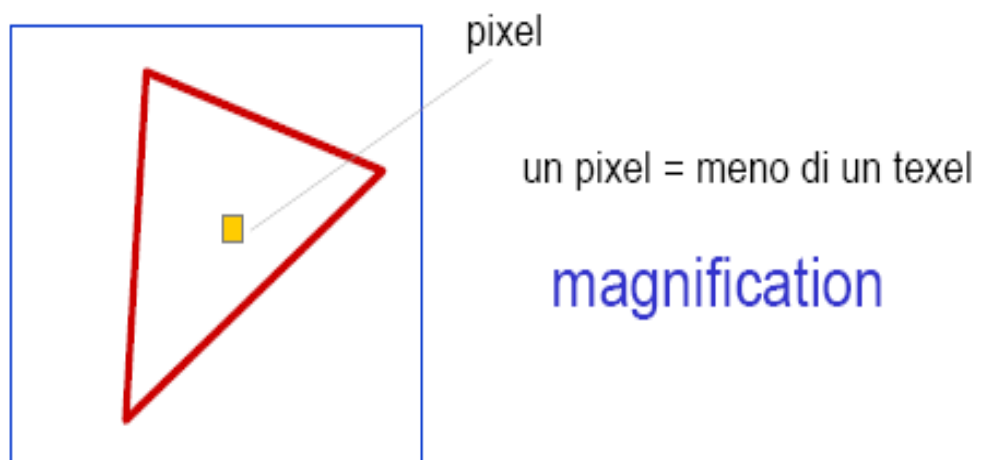
Cosa accade se un pixel copre molti texel?

Minification: la texture deve essere rimpicciolita per avere meno texel.

Un pixel corrisponde a molti texel. Avendo molti texel, si dovrebbero vedere molti dettagli. Ma poiché ad ogni pixel contribuisce un solo texel (o una media), vuol dire che ci sono texel non utilizzati.



(a) Minification



Specificare i parametri

- `void TexParameter{if v} ( enum target, enum pname, T params );`

dove: -**target** la texture per cui specificare i parametri
-**pname** il parametro da impostare
-**params** valori del parametro

Modifica dei parametri per l'ingrandimento e la riduzione:

glTexParameterf

- **pname** specifica quale filtro modificare
 - **GL_TEXTURE_MAG_FILTER** oppure **GL_TEXTURE_MIN_FILTER** (obbligatorio)
- **param** specifica se è più importante la velocità o la qualità dell'immagine
 - **GL_NEAREST** per scegliere il texel più vicino alle coordinate texture calcolate per il pixel.
 - **GL_LINEAR** per utilizzare la *media pesata* dei quattro texel più vicini alla coordinate texture calcolate per il pixel.

Abilitare il Texture Mapping

```
void glEnable( GLenum mode )
```

- ***mode*** deve essere uno tra:
GL_TEXTURE_1D,
GL_TEXTURE_2D,
GL_TEXTURE_3D_EXT

Esercizio 1

- Disegnare un pesce che nuota seguendo un percorso sinusoidale.
- Il punto di vista deve essere posizionato vicino al centro del tracciato del percorso e lo sguardo deve seguire il pesce
- Caricare l'immagine fish.rgb
- Utilizzare glutIdleFunc per animare il supporto dell'immagine

Esempio: texture su quad

```
int main( int argc, char *argv[] )
{
    char    *imageFileName = "fish.rgb";
    unsigned *image;
    GLsizei width=640, height=480;
    GLsizei imageWidth, imageHeight, components;
    GLdouble aspect;

    glutInit( &argc, argv );

    image = read_texture(imageFileName, &imageWidth,
        &imageHeight, &components);

    glutInitWindowSize( width, height );
    glutInitDisplayMode( GLUT_RGBA | GLUT_DOUBLE );
    glutCreateWindow( argv[0] );

    initTexture( image, imageWidth, imageHeight );
    initgfx();

    aspect = (GLdouble) width / (GLdouble) height;
    glMatrixMode( GL_PROJECTION );
    glLoadIdentity();
    gluPerspective( 45.0, aspect, 1.0, 50.0 );
    glMatrixMode( GL_MODELVIEW );
    glLoadIdentity();
    glTranslatef( 0.0, 0.0, -12.0 );

    glutDisplayFunc( drawScene );

    glutMainLoop();
}

GLvoid initgfx( void )
{
    glClearColor( 0.0, 0.0, 0.0, 1.0 );
}
```

```
GLvoid initTexture( unsigned *image, GLsizei imageWidth, GLsizei
    imageHeight )
{
    GLsizei sWidth=256, sHeight=256;
    unsigned *sImage;

    glTexParameterf(GL_TEXTURE_2D,
        GL_TEXTURE_MIN_FILTER, GL_NEAREST);

    sImage = (unsigned *)malloc( sWidth*sHeight*4*sizeof( unsigned ) );
    gluScaleImage( GL_RGBA, imageWidth, imageHeight,
        GL_UNSIGNED_BYTE, image,
        sWidth, sHeight, GL_UNSIGNED_BYTE, sImage );

    glTexImage2D(GL_TEXTURE_2D, 0, GL_RGBA, sWidth, sHeight,
        0, GL_RGBA, GL_UNSIGNED_BYTE, sImage);

    glEnable( GL_TEXTURE_2D );
}

GLvoid drawScene(void)
{
    float v0[3] = { -1.5, -1.0, 0.0 };
    float v1[3] = { 1.5, -1.0, 0.0 };
    float v2[3] = { 1.5, 1.0, 0.0 };
    float v3[3] = { -1.5, 1.0, 0.0 };

    float t0[2] = { 0.0, 0.0 };
    float t1[2] = { 1.0, 0.0 };
    float t2[2] = { 1.0, 1.0 };
    float t3[2] = { 0.0, 1.0 };

    glClear( GL_COLOR_BUFFER_BIT );

    glColor3f( 1.0, 1.0, 1.0);

    glBegin( GL_QUADS );
        glTexCoord2fv( t0 ); glVertex3fv( v0 );
        glTexCoord2fv( t1 ); glVertex3fv( v1 );
        glTexCoord2fv( t2 ); glVertex3fv( v2 );
        glTexCoord2fv( t3 ); glVertex3fv( v3 );
    glEnd();
    glutSwapBuffers();
}
```

Texture Multiple

`void glGenTextures (GLsizei n, GLuint *texnames)`

- Genera texture names (codici di identificazione)

`void glBindTexture (GLenum target, GLuint texname)`

- Crea una nuovo oggetto texture (*target*) e lo assegna ad un texture con nome (la prima volta)
- Assegna ad un oggetto texture una texture con nome (bind)
- Chiamata con *texname*=0 smette di utilizzare oggetti texture (unbind)

`void glPrioritizeTextures (GLsizei n, const GLuint *texnames, const GLclampf *priorities)`

- Imposta la priorità di residenza della texture (per velocità)

`void glDeleteTextures (GLsizei n, GLuint *texnames )`

- Cancella oggetti texture

Nella versione 1.0 di OpenGL le texture multiple sono supportate tramite l'estensione EXT_texture_object

Esempio

```
static GLuint texnames[2];

/* generate unused texture names */
glGenTextures(2, texnames);

/* bind, then define, each texture */
glBindTexture(GL_TEXTURE_2D, texnames[0]);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexImage2D(GL_TEXTURE_2D, 0, 4, 64, 64, 0, GL_RGBA, GL_UNSIGNED_BYTE, redtex);
glBindTexture(GL_TEXTURE_2D, texnames[1]);
glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
glTexImage2D(GL_TEXTURE_2D, 0, 4, 64, 64, 0, GL_RGBA, GL_UNSIGNED_BYTE, greentex);

...
glBindTexture(GL_TEXTURE_2D, texnames[1]);
drawTexturesPolygon(); /* uses greentex */
glBindTexture(GL_TEXTURE_2D, texnames[0]);
drawTexturesPolygon(); /* uses redtex */
```

Esercizio 2

- Disegnare due pesci che nuotano seguendo due percorsi sinusoidali e tra loro perpendicolari
- Il punto di vista deve essere posizionato abbastanza lontano da vedere entrambi i pesci
- Caricare l'immagine fish.rgb
- Caricare l'immagine fisha.rgb

Generazione automatica di coordinate texture

OpenGL ha tre tipi di coordinate texture che possono essere generate automaticamente:

- `GL_OBJECT_LINEAR` : l'immagine texture rimane fissata ad un oggetto
- `GL_EYE_LINEAR` : un oggetto che si muove appare come se nuotasse nella texture
- `GL_SPHERE_MAP` : utilizzato per il mapping di ambienti

Generazione automatica di coordinate texture 2D

- `glTexGeni( GL_S, GL_TEXTURE_GEN_MODE, mode );`
- `glTexGeni( GL_T, GL_TEXTURE_GEN_MODE, mode);`

Specificano il *mode* tra `GL_OBJECT_LINEAR`, `GL_EYE_LINEAR`, `GL_SPHERE_MAP`.

- `glEnable( GL_TEXTURE_GEN_S );`
- `glEnable( GL_TEXTURE_GEN_T );`

Abilita la generazione delle coordinate S e T.

- `static GLfloat sgenparams[ ] = { 1, 0, 0, 0 };`
- `static GLfloat tgenparams[ ] = { 0, 1, 0, 0 };`
- `glTexGenfv( GL_S, GL_OBJECT_PLANE, sgenparams );`
- `glTexGenfv( GL_T, GL_OBJECT_PLANE, tgenparams );`

Per genera le coordinate texture basandosi sui piani **sgenparams** e **tgenparams** (modalità più generica).

Esercizio 3

- Disegnare una sfera che rotola con dei pesci disegnati sopra
- Caricare l'immagine fish.rgb
- Utilizzare la primitiva glut per la sfera

Livelli di dettaglio multipli

- Per alleggerire il carico computazionale per l'applicazione delle texture, si possono definire delle texture a definizione variabile chiamate *mipmap*.
- Quando un oggetto è visualizzato come lontano, viene utilizzata una immagine a bassa risoluzione
- Quando un oggetto è visualizzato come vicino, viene utilizzata una immagine ad alta risoluzione
- Le mipmap sono le immagini da utilizzare come una unica texture ed hanno dimensioni uguali alle potenze di 2 comprese tra la dimensione massima che può assumere la texture e quella minima, cioè 1x1.

Creazione automatica di mipmap

- GLint **gluBuild2DMipmaps**(GLenum *target*, GLint *internalformat*, GLsizei *width*, GLint *height*, GLenum *format*, GLenum *type*, void **data*)

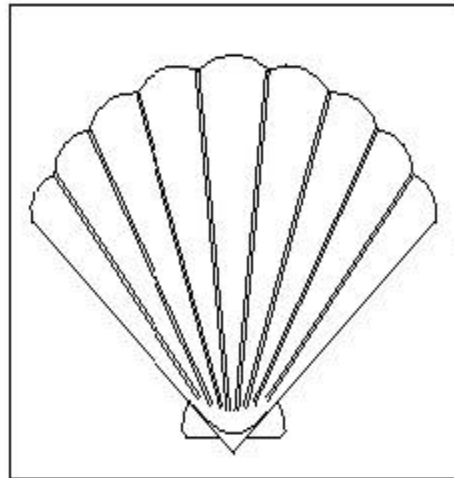
dove:

- ***target*** il tipo di texture
- ***internalformat*** formato di memorizzazione interno della texture (deve essere 1, 2, 3, 4, o una delle costanti simboliche)
- ***width*** larghezza dell'immagine texture
- ***height*** altezza dell'immagine texture
- ***format*** formato dei dati pixel
- ***type*** tipo dei dati pixel
- ***data*** puntatore all'immagine in memoria

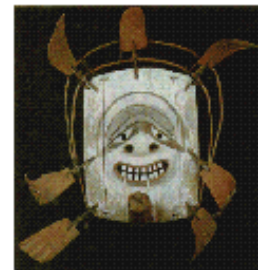
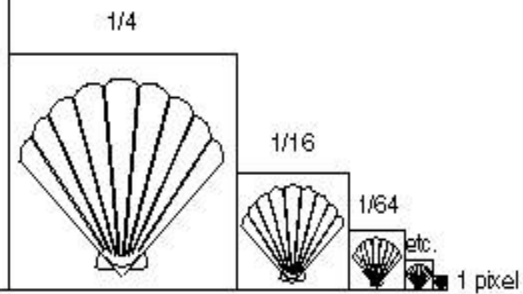
GLU esegue automaticamente la scalatura dell' immagine al fine di ottenere potenze di 2.

Esempi

Original Texture



Pre-Filtered Images



Creazione manuale di mipmap

- Caricare tutte le immagini dalla più grande alla più piccola (con dimensione 1x1)
- Utilizzare più volte `glTexImage2D` per definire i vari livelli della mipmap (secondo parametro di input).
Il livello 0 corrisponde all'immagine con risoluzione maggiore, il livello più alto all'immagine di dim. 1x1

Parametri di riduzione con mipmap

- `void glTexParameterf(target,
GL_TEXTURE_MIN_FILTER, param)`

dove ***params*** può essere:

- **GL_NEAREST_MIPMAP_NEAREST** usa il texel più vicino alla coordinata texture calcolata.
- **GL_LINEAR_MIPMAP_NEAREST** usa la media pesata dei quattro texel più vicini

I filtri seguenti utilizzano le due mipmaps più vicine:

- **GL_NEAREST_MIPMAP_LINEAR** (default) usa la media pesata tra i texel più vicini alle coordinate texture calcolate in ogni mipmap
- **GL_LINEAR_MIPMAP_LINEAR** usa la media pesata dei 4 texel più vicini alle coordinate texture calcolate in ogni mipmap

Esercizio 4

- Disegnare un pesce fermo che può essere visualizzato a distanza variabile.
- Rendere possibile la modifica del filtro per mipmap tra i quattro valori

Texture Environment

- Serve a definire la modalità di composizione dei colori delle texture e dei poligoni di supporto.
- Chiamiamo:
- C_p le componenti RGB del poligono
- C_t le componenti RGB della texture
- A_p e A_t le componenti alpha del poligono e della texture
- C_r e A_r le componenti risultanti dopo la composizione

Texture Environment

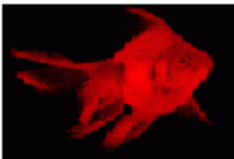
C_p



C_t

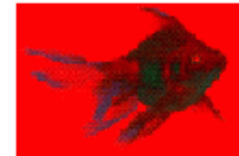


Modulate (default)



$$C_r = C_t * C_p$$
$$A_r = A_t * A_p$$

Blend



$$C_r = (1 - C_t) * C_p + C_t * C_c$$
$$A_r = A_t * A_p$$

Decal



$$C_r = (1 - A_t) * C_p + A_t * C_t$$
$$A_r = A_p$$

Replace



$$C_r = C_t$$
$$A_r = A_t$$

Impostazione dell' Environment

- `void glTexEnvf (GL_TEXTURE_ENV ,
GGLenum pname, const GLfloat param)`

Se ***pname*** è **GL_TEXTURE_ENV_MODE**:

param può essere:

GL_MODULATE (default), ***GL_DECAL***, ***GL_BLEND*** oppure
GL_REPLACE

Se ***pname*** è **GL_TEXTURE_ENV_COLOR**:

param è il colore RGBA che viene utilizzato
per il BLEND (C_c)

Esercizio 5

- Creare un corridoio con il soffitto bianco, le pareti di mattoni ed il pavimento blu di piastrelle.
- Applicare alle pareti degli applique di tessuto viola con una luce all'interno
- Permettere all'osservatore di muoversi nel corridoio

Altri Mapping

Esistono diverse tecniche associate al texture mapping tipicamente adoperate per aumentare il fotorealismo di una scena. Alcuni esempi:

- Environment Mapping
- Bump Mapping
- Displacement Mapping
- Projective Texture Mapping
- Multitexturing

Environment Mapping

L'environment mapping è usato per simulare la diffusione della luce incidente di un ambiente su di una superficie

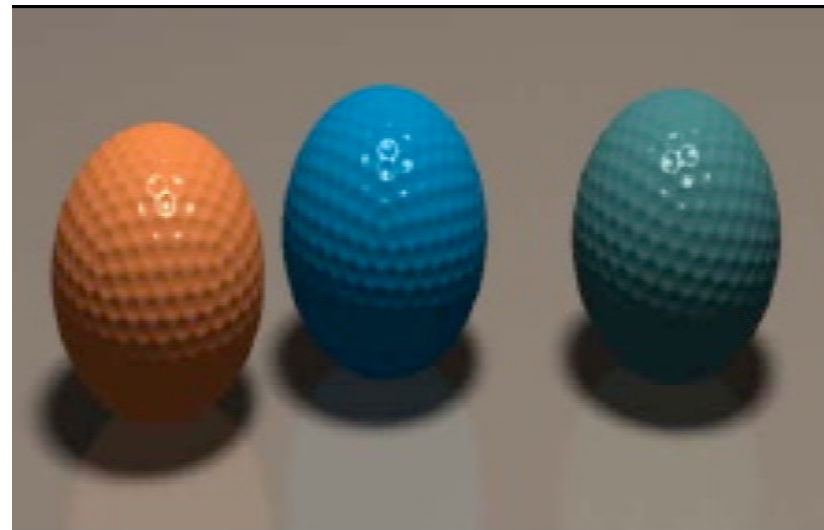
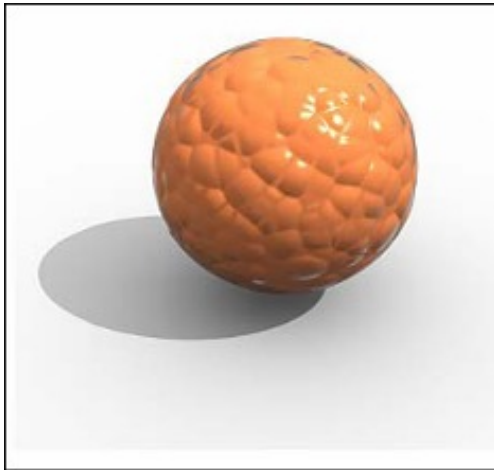
- Utile per superfici lucide



Bump Mapping

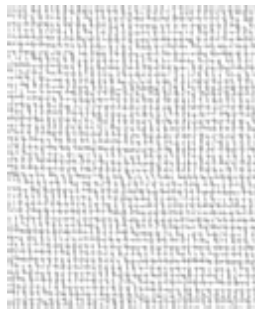
Il *Bump Mapping* è un metodo per simulare superfici scabre, con rilievi (quindi tridimensionali), utilizzando textures bidimensionali

– Ad esempio in un oggetto che rappresenta un tronco di legno, le fenditure della corteccia, grazie al bump mapping, possono apparire tridimensionali anche se non sono realizzate realmente con poligoni.



Bump Mapping

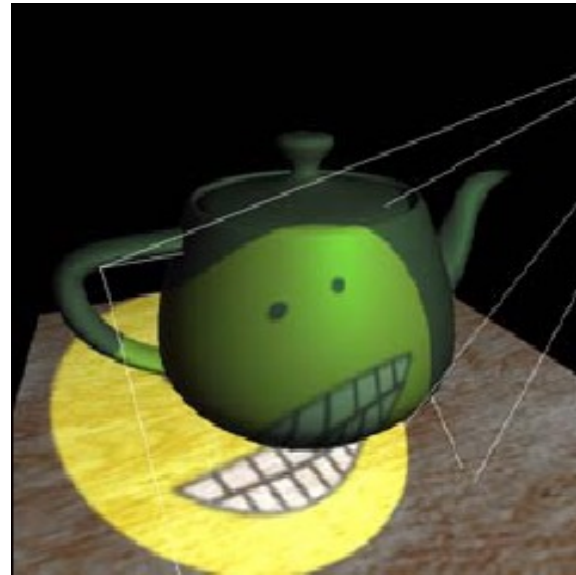
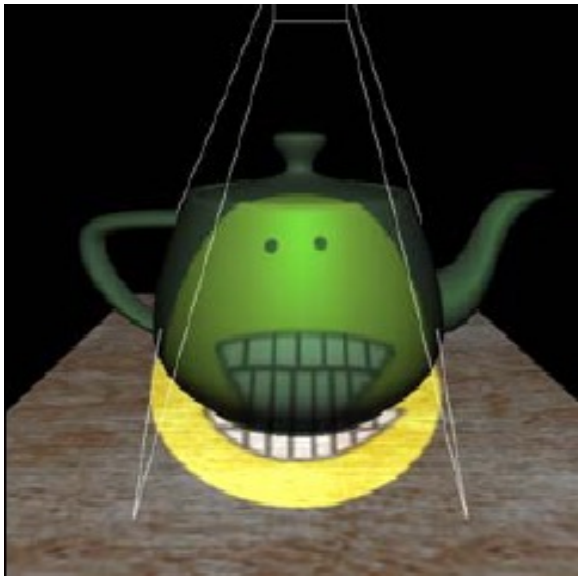
Si realizza perturbando la normale in un punto con il valore corrispondente nella texture; siccome il modello locale di illuminazione usa la normale per calcolare le intensità di colore, il risultato di un bump mapping è di alterare lo shading della superficie (senza modificare ovviamente la geometria)



Projective Texture Mapping

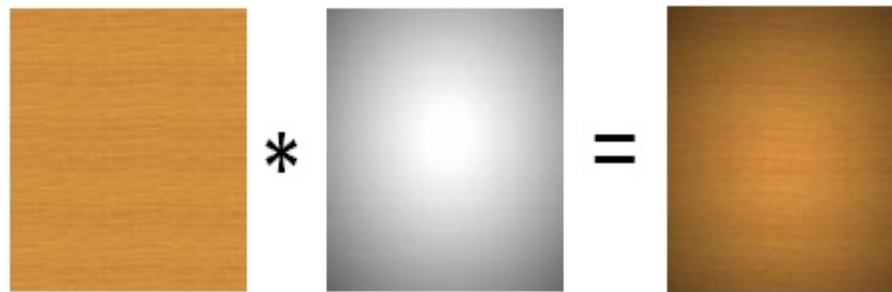
Il Projective Texture Mapping è un metodo per la proiezione di texture su una o più superfici

– Esempi: luci/ombre



Multitexturing

- Il *Multitexturing* consiste nell'applicazione di due o più texture su di una superficie in una singola passata
 - Le texture sono combinate attraverso qualche operazione
 - L'hardware è in grado di gestire anche 8 texture in un singolo passo
 - Esempi: lightmapping, decals



Organizzazione di una texture

- Solitamente l'insieme delle texture di un singolo oggetto sono organizzate in un unico file
- Ogni porzione dell'immagine mappa una parte della mesh

